claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-ac4a70d256765d879.jsonl`
- SHA-256: `74be5dee425a9e372952a620a8e4561102198e8d5e3dbe70e53d265b9465cccb`
- Source modified: `2026-06-22T15:04:22+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T15:03:09.424Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: robustness-security
Severity: minor
Title: Per-job temp directory from mkdtemp('cr-ingest-') is never cleaned up
File: backend/main.py:449
Detail: _ingest_remote_segments creates `tempfile.mkdtemp(prefix='cr-ingest-')` to fetch intervals.csv and never removes it. One small leaked directory (the CSV) accumulates per cloud-dispatched job, relying solely on OS temp reclamation (which on Windows/long-lived servers may never happen). On a long-running serving box this is unbounded inode/file growth. Grep confirms no rmtree/TemporaryDirectory cleanup for this prefix. (A sibling pre-existing leak exists at backend/compute/cloud_run.py:148 with prefix 'cr-read-' for report.json/marker reads ? same pattern, out of this PR's diff but worth folding into the same fix.)
Fix: Use `with tempfile.TemporaryDirectory(prefix='cr-ingest-') as d:` and read the CSV inside the block, or rmtree the dir in a finally. Apply the same to cloud_run._read_json's 'cr-read-' dir while you're there.

2. assistant (2026-06-22T15:03:12.960Z)

I'll verify this finding by examining the cited code in the bhi-cloudrun repository.

3. user (2026-06-22T15:03:15.737Z)

```
 backend/main.py                  | 100 ++++++++++++++++++
 tests/test_api_cloud_dispatch.py | 212 ++++++++++++++++++++++++++++++++++++++++
 2 files changed, 312 insertions(+)
```

4. assistant (2026-06-22T15:03:18.963Z)

The branch only modifies `backend/main.py` and a test file. Let me look at the diff for `main.py` around the cited line.

5. user (2026-06-22T15:03:20.395Z)

```
diff --git a/backend/main.py b/backend/main.py
index 4e5a19a..04ffb67 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -1,10 +1,12 @@
 import os
 import sys
```

```
 import argparse
+import csv
 import json
 import logging
 import shutil
 import socket
+import tempfile
 import threading
 import uuid
 import webbrowser
@@ -436,6 +438,94 @@ def download_reel(filename: str):
     return FileResponse(path, media_type=media, filename=name)


+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
(cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                    "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                           play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
+    == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
``None``
+    for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
+    from backend.compute import ComputeJobSpec, get_compute_backend
+
+    compute = get_compute_backend(global_config)
+    if compute is None:
+        return None  # default-OFF ? run the in-process segmenter below (unchanged)
```

```
+
+    storage_cfg = getattr(global_config, "storage", None)
+
+    def _failed(msg: str) -> Dict[str, Any]:
+        # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+        db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+        return {"video_id": video_id, "status": "failed", "message": msg,
+                "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+    # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+    try:
+        input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+    except ValueError as e:
+        return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+    if input_ref.backend == "local":
+        return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+                       "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+    if not out_root:
+        return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+                       "? that's where the Cloud Run job writes the result.")
+
+    notify("dispatching (cloud_run)")
+    logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+    spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+    try:
+        result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
+    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+        logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+        return _failed(f"cloud dispatch error: {e}")
+    if result.returncode != 0:
+        return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+    notify("ingesting (cloud_run)")
+    storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+    _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+    # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+    segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+    _finalize_reingest(video_id, segments, play_wm, cand_wm)
+    return {"video_id": video_id, "status": "success",
+            "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+            "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+
 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
     """The full hash ? validate ? segment ? report pipeline for one video.

@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
         # if the run yields segments; otherwise drop the new partial rows and keep the old.
         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+        # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
```

```
+        # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+        # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+        # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+        # they're dropped on the cloud path in this slice ? a documented follow-up.)
+        remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+        if remote is not None:
+            segmentation_ran = True
+            response = remote
+            return response
         segmenter = segmenter_cls(db_instance, global_config)
         try:
             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
```

6. assistant (2026-06-22T15:03:23.291Z)

Now let me confirm the exact line number of the `mkdtemp` call and verify there's no cleanup in the function.

7. user (2026-06-22T15:03:23.837Z)

```
441     def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) ->
int:
442         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
443         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
444         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
445         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
446         backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
447         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
448         ref = storage.parse_uri(uri)
449         dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or
"intervals.csv")
450         local = storage.fetch(uri, dst, config=storage_config)
451         n = 0
452         with open(local, newline="", encoding="utf-8") as f:
453             for row in csv.DictReader(f):
454                 try:
455                     start, end = float(row["start"]), float(row["end"])
456                 except (KeyError, TypeError, ValueError):
457                     continue
458                 meta: Dict[str, Any] = {"source": "cloud_run",
459                                         "ending_reason": (row.get("ending_reason") or
"").strip()}
460                 sc = (row.get("shot_count") or "").strip()
461                 if sc:
462                     try:
463                         meta["shot_count"] = int(float(sc))
464                     except ValueError:
465                         meta["shot_count"] = sc
466                 db_instance.add_play_segment(video_id, start, end, metadata=meta)
467                 n += 1
468         return n
469
470
```

8. assistant (2026-06-22T15:03:26.866Z)

Confirmed: line 449 creates `tempfile.mkdtemp(prefix="cr-ingest-")` and the function returns at line 468 with no cleanup. Let me verify the claims about no cleanup anywhere for this prefix, and check the sibling `cloud_run._read_json` pattern referenced.

9. user (2026-06-22T15:03:28.360Z)

```
449:    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
```

10. assistant (2026-06-22T15:03:31.301Z)

No `rmtree`/`TemporaryDirectory`/cleanup for `cr-ingest-` anywhere in `main.py`. Let me verify the sibling claim at `cloud_run.py:148` and check the overall pattern in the compute module.

11. user (2026-06-22T15:03:32.394Z)

```
148:            dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json")
```

12. user (2026-06-22T15:03:34.962Z)

```
138             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139                 return None
140             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
141
142         def _read_json(self, uri: str) -> dict:
143             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146             try:
147                 ref = parse_uri(uri)
148                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149                 local = fetch(uri, dst, config=self._storage_config)
150                 with open(local, "r", encoding="utf-8") as f:
151                     data: Any = json.load(f)
152                 return data if isinstance(data, dict) else {}
153             except Exception:
154                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155                 return {}
156
```

13. assistant (2026-06-22T15:03:38.660Z)

Both claims are confirmed. Now let me assess the severity accurately. The key question for robustness-security severity: how often is `_read_json` (the `cr-read-` leak) called? It's the bucket-poll, so it runs on every poll iteration during job completion. Let me check how the polling uses it.

14. user (2026-06-22T15:03:39.893Z)

```
1:"""``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and bucket-poll
7:  2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16
? the
10:     cold-start is absorbed by the async poll.
33:from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
57:        execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
83:    """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
106:        done_uri = f"{out_root}/_dispatch/{token}.done"
108:                            "--done-uri", done_uri]
```

```
116:        LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117:                 self._job_name, execution, done_uri)
119:        marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
130:        report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
134:    # --- storage-backed completion signal (bucket-poll) ---
135:    def _read_marker(self, done_uri: str) -> Optional[dict]:
136:        """The done-marker dict if the worker has written it, else None ? poll_until keeps
waiting."""
137:        ref = parse_uri(done_uri)
140:        return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
142:    def _read_json(self, uri: str) -> dict:
```

15. assistant (2026-06-22T15:03:43.270Z)

Now I understand the full picture. Let me confirm `_read_marker` (which calls `_read_json`) is invoked once per poll iteration via `poll_until`, plus a final `report.json` read. So during a single job, `cr-read-` directories leak once per poll AND `cr-ingest-` leaks once. Let me check the poll interval to gauge magnitude.

16. user (2026-06-22T15:03:44.362Z)

```
7:- ``poll_until`` ? a bounded poll loop for an in-progress external op (``poll_every_n_sec``
cadence,
70:DEFAULT_POLICY = ExecutionPolicy()
78:def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
125:def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy = DEFAULT_POLICY,
*,
```

17. user (2026-06-22T15:03:47.079Z)

```
40          """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
41
42
43      @dataclass(frozen=True)
44      class ExecutionPolicy:
45          """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46          mode: WaitMode = WaitMode.WAIT_AND_RESUME
47          poll_every_n_sec: float = 10.0
48          op_timeout_sec: float = 120.0         # HARD per-attempt deadline ? the hang-killer
that was missing
49          max_wait_sec: float = 1800.0          # total wall budget for a poll loop
50          max_retries: int = 5                  # retries (=> max_retries+1 attempts) for
failures/hangs
51          backoff_base_sec: float = 3.0         # exponential: backoff_base * 2**attempt
52          on_timeout: str = "reschedule"        # reschedule | partial | fail  (consumed by
P2/P3)
53          resumable: bool = True
54
55          def to_dict(self) -> dict:
56              d = asdict(self)
57              d["mode"] = self.mode.value
58              return d
59
60          @classmethod
61          def from_dict(cls, d: dict) -> "ExecutionPolicy":
62              known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
63              kw = {k: v for k, v in d.items() if k in known}
64              if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65                  kw["mode"] = WaitMode(kw["mode"])
66              return cls(**kw)
67
```

```
68
69      #: The sane default (also what a job gets if it declares no policy).
```

18. assistant (2026-06-22T15:03:51.125Z)

This sharpens the magnitude. `poll_every_n_sec=10.0`, `max_wait_sec=1800.0` ? up to ~180 poll iterations per job. But note: `_read_marker` first calls `exists()` (line 138) and only calls `_read_json` once the marker is present (i.e. once the job is done). Let me re-read that to be precise.

19. user (2026-06-22T15:03:51.610Z)

```
119             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                                 label="cloud-run-infer", logger=LOG)
121             video_id = str(marker.get("video_id", ""))
122             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124             rc = int(marker.get("returncode", 1))
125             if not video_id:
126                 LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                             "video_id) ? treating as failure", self._job_name)
128                 rc = rc or 1
129             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:
136             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137             ref = parse_uri(done_uri)
138             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139                 return None
140             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
```

20. assistant (2026-06-22T15:04:06.209Z)

Now the precise picture is clear:

- `_read_marker` short-circuits on `exists()` returning False, so `_read_json` (the `cr-read-` leak) fires only when the marker is present (1×) plus the final `report.json` read (1×) ? **2 `cr-read-` dirs per job**, not one-per-poll.
- `_ingest_remote_segments` leaks **1 `cr-ingest-` dir per job**.

So roughly **3 leaked temp directories per cloud-dispatched job** (each holding one small file). The finding's core claim holds: the `cr-ingest-` dir at main.py:449 is genuinely never cleaned up, the grep confirms no cleanup, and the sibling `cr-read-` leak at cloud_run.py:148 is real too.

Now the severity assessment. Let me weigh it honestly:

- It's a real, verifiable resource leak (not a false alarm). Each job leaks ~3 tiny dirs.
- The growth is genuinely unbounded across process lifetime on a long-running serving box, relying solely on OS temp reclamation. On Linux, `/tmp` is typically cleared on reboot or by `systemd-tmpfiles` (default 10 days), and Cloud Run instances are ephemeral/recycled ? so in the actual deployment target (Cloud Run serving), the leak is largely self-limiting. But the engine is also the locally-installed `indexer --app` long-lived box per the project notes, where it can

persist.
- Magnitude is small: a few empty-ish dirs with one small CSV/JSON each per job. It's not a fast-fill or DoS vector. Inode exhaustion would require an enormous number of jobs.
- The fix is trivial and correct (`TemporaryDirectory` context manager / `finally` rmtree).

This is a legitimate but low-impact hygiene issue. The finding's reasoning is sound and the cited code holds up exactly. The severity of "minor" is appropriate ? it's a real defect (worth fixing, leak is unbounded by process lifetime) but low-impact (tiny per-leak footprint, partially mitigated by ephemeral Cloud Run instances and OS temp reclamation, no security/correctness consequence). It's slightly above a pure nit because the leak is genuinely unbounded and the fix touches the actual new code in this PR.

One correction to the finding's framing: it says "One small leaked directory (the CSV)" per job for the in-PR code ? accurate for `cr-ingest-`. The sibling note is correctly flagged as pre-existing/out-of-diff.

21. user (2026-06-22T15:04:14.476Z)

Structured output provided successfully

22. assistant (2026-06-22T15:04:22.008Z)

Verified the finding against the cited code. It holds up as a real, low-impact defect.

Summary:
- `C:/Users/avidu/Projects/bhi-cloudrun/backend/main.py:449` ? the new `_ingest_remote_segments` calls `tempfile.mkdtemp(prefix="cr-ingest-")` and returns at line 468 with no cleanup. Grep confirms no `rmtree`/`TemporaryDirectory` for that prefix. One temp dir (holding `intervals.csv`) leaks per cloud-dispatched job.
- Sibling claim also confirmed: `C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/cloud_run.py:148` (`_read_json`, prefix `cr-read-`) has the same pattern ? pre-existing, out of this PR's diff. It leaks ~2 dirs per job (fires after `exists()` is true, plus the final `report.json` read), not one-per-poll, so the finding doesn't overclaim.

is_real=true, severity=minor. It's a genuine unbounded-over-process-lifetime leak with a trivial correct fix (`with tempfile.TemporaryDirectory(...)`), but per-leak footprint is tiny, there's no security/correctness consequence, and the ephemeral Cloud Run serving target plus OS temp reclamation partially self-limit it ? above a nit because the leaking line is new code in this diff and growth is genuinely unbounded.